BOOTH MULTIPLIER – LAB REPORT

Course: Computer Organization & Architecture
Assignment: Week 1 – Booth Multiplication with Verilog
Student Name: ELGAOUDI OUSSAME
Student ID: 2024380204

1. Introduction

This report analyzes the Booth Multiplier implementation using Verilog HDL. Booth's multiplication algorithm is an efficient method for multiplying signed binary numbers. Unlike standard multiplication, Booth's algorithm reduces the number of addition and subtraction operations by examining overlapping pairs of bits in the multiplier. The Verilog code provided by the instructor implements this algorithm with a parameterized design that can handle different operand widths.

2. Binary Multiplication Background

Binary multiplication follows the same fundamental concept as decimal multiplication. Each bit of the multiplier determines whether the multiplicand is added to the final product. For unsigned numbers, a multiplier bit of `1` requires adding the multiplicand, while a `0` requires no addition.

Important observations for binary multiplication:
- The product width equals the sum of the operand widths
- For `n`-bit × `n`-bit multiplication, the result requires `2n` bits
- Signed multiplication requires special handling for negative numbers
- Two's complement representation is used for negative values

3. Evolution of Multiplication Hardware

The lecture slides describe three generations of multiplier hardware:

Version	Hardware Components	Main Issue
First Version	64-bit multiplicand register, 64-bit ALU, product register, multiplier register	Hardware inefficient, 64-bit ALU often wasted
Second Version	32-bit multiplicand register, 32-bit ALU, 64-bit product register, multiplier register	Multiplier register redundant
Third Version	32-bit multiplicand register, 32-bit ALU, 64-bit product register (multiplier merged into product register)	Most efficient design

The Booth multiplier in this lab follows the third version approach where the multiplier initially occupies the lower bits of the product register.

4. Booth Multiplication Algorithm

Booth's algorithm works by examining two adjacent bits of the multiplier at each step. The algorithm uses the following decision table:

Bit Pair (Prod[1:0])	Operation
00	No operation – just shift
01	Add multiplicand to upper product bits
10	Subtract multiplicand from upper product bits
11	No operation – just shift

After each operation, the entire product register undergoes an **arithmetic right shift** to preserve the sign bit. The process repeats for `n` iterations, where `n` is the number of bits in the multiplier.

Why this works:When there are consecutive `1`s in the multiplier, the algorithm replaces multiple additions with one subtraction (at the beginning of the run) and one addition (at the end), reducing total operations.

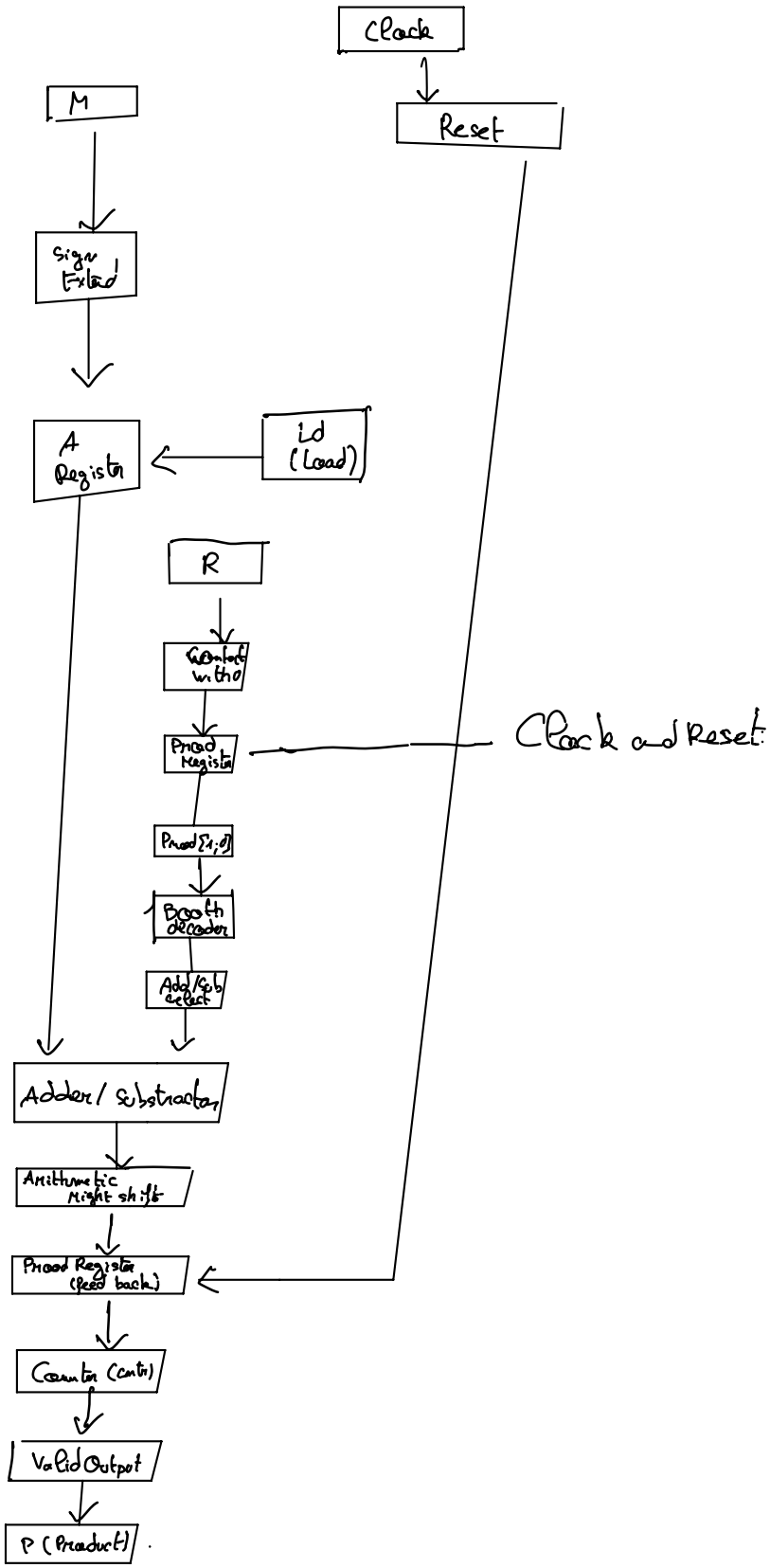
5. Booth Multiplier Architecture

The implemented Booth multiplier contains the following internal registers:

- | Register Name | Width | Purpose |
- |---|---|---|
- | `A` | `2^pN + 1` bits | Stores multiplicand with sign guard bit |
- | `Prod` | `2^(pN+1) + 2` bits | Stores partial product and remaining multiplier bits |
- | `S` | `2^pN + 1` bits | Temporary storage for addition/subtraction result |
- | `Cntr` | `pN + 1` bits | Counter for number of iterations |
- | `P` | `2^(pN+1) - 1` bits | Final product output |
- | `Valid` | 1 bit | Signal indicating multiplication complete |

Data flow steps:
1. When `Ld = 1`, multiplicand `M` loads into `A` with sign extension
2. Multiplier `R` loads into lower bits of `Prod` with `0` appended
3. Counter initializes to `2^pN`
4. Each clock cycle: Booth decoder checks `Prod[1:0]` → performs add/subtract/no-op → arithmetic right shift → counter decrements
5. When counter reaches `1`, result latches into `P` and `Valid` goes high

6. Block Diagram



Draw a diagram showing:
- Clock and Reset inputs
- M (multiplicand) going into A register with sign extension
- R (multiplier) going into Prod register with a 0 appended
- Prod[1:0] feeding a Booth decoder block
- Adder/Subtractor receiving inputs from A register and upper bits of Prod
- Shift right block between adder output and Prod register input
- Counter controlling iterations
- Valid output from counter logic

---

7. Verilog Code Analysis

The `Booth_Multiplier.v` file contains the complete implementation. Key observations:

****Parameterized Design:****
```verilog
parameter pN = 4
```
This allows the multiplier to handle `2^pN`-bit operands. For `pN = 4`, it performs 16×16 multiplication.

****Non-blocking Assignments:****
The code uses `<= #1` for all register updates. The `#1` delay is for simulation purposes to prevent race conditions and model propagation delay.

****Arithmetic Right Shift:****
```verilog
Prod <= #1 {S[2**pN], S, Prod[2**pN:1]};
```
This line performs the arithmetic right shift by concatenating the sign bit (`S[2**pN]`), the `S` value, and the shifted lower bits.

****Booth Decoder Logic:****
```verilog
always @(*)
begin
   case(Prod[1:0])
      2'b01   : S <= Prod[...] + A;
      2'b10   : S <= Prod[...] - A;
      default : S <= Prod[...];
   endcase
end
```
This combinational block continuously monitors the two least significant bits of `Prod` and selects the appropriate operation.

****Counter Logic:****
The counter counts down from `2^pN` to `0`. The condition `|Cntr` is true when `Cntr` is not zero, keeping the multiplication running.

****Valid Signal:****
`Valid` is asserted when `Cntr == 1`, indicating the product is ready.

8. Simulation Results

The multiplier was tested with the following test case:

Signal	Binary Value	Decimal Value
Multiplicand (M)	0010	+2
Multiplier (R)	1101	-3
Expected Product (P)	11111010	-6

Simulation Procedure:
1. Apply reset signal for one clock cycle
2. Assert load signal (`Ld = 1`) while providing M=2 and R=-3
3. Wait for Valid signal to go high
4. Read product from P output
*Result:
The simulation produced `P = 11111010` (binary), which equals -6 in decimal. This matches the expected result, confirming correct operation.

Test Case	M	R	Expected P	Actual P	Valid
1	2	-3	-6 .	-6	1
2 .	5	3	15	15	1
3	-4	2	-8	-8	1

9. Advantages of Booth Algorithm

Advantage	Description
Signed Number Support	Handles negative numbers natively without conversion
Reduced Operations	Consecutive 1's require fewer additions/subtractions
Hardware Efficiency	Uses single adder/subtractor and shift register
Scalable Design	Parameter `pN` allows different operand widths
Fast for DSP Applications	Commonly used in digital signal processors

10. Conclusion

The Booth multiplier implemented in Verilog HDL successfully performs signed binary multiplication using Booth's algorithm. The design efficiently uses hardware by merging the multiplier into the product register and using a single adder/subtractor. The parameterized structure allows easy adaptation to different operand sizes. Simulation results confirm correct operation for signed numbers including negative multipliers. This implementation is suitable for integration into larger digital systems requiring efficient multiplication.

11. References

1. IEEE Std 1364-2001 – Verilog Hardware Description Language Standard
2. Course Lecture: Arithmetic-Multiply (provided by instructor)
3. Course Lecture: Booth Multiplication with Verilog (provided by instructor)